

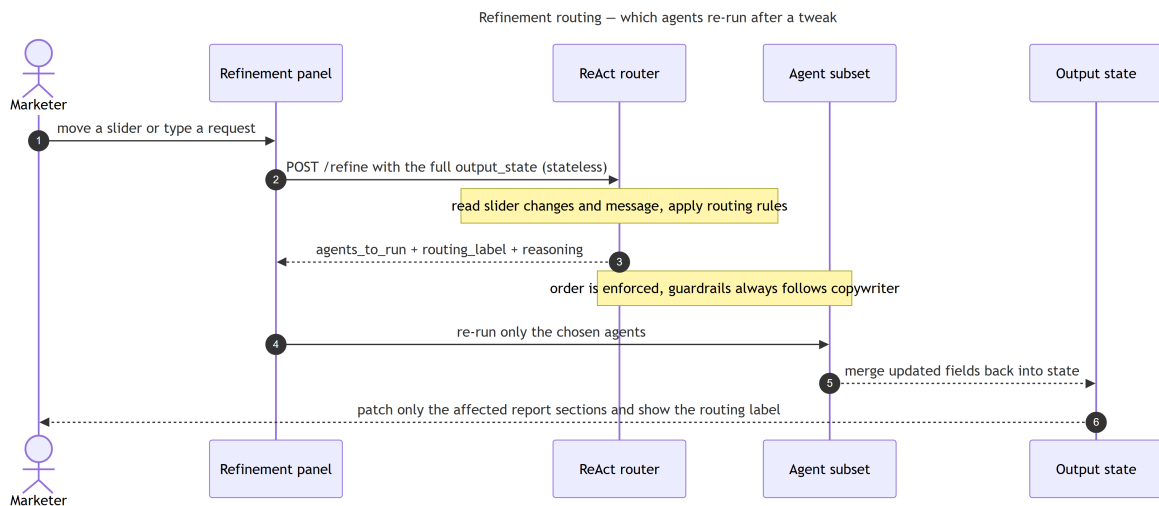
MARKETING AGENT — THE REFINEMENT LOOP

How a user improves a campaign without re-running everything

WHAT THIS IS

The first campaign is rarely the last word. The refinement loop lets a user adjust the result — change the tone, shift the audience age, ask for a more urgent call to action — and get an updated campaign back in seconds, without paying for the full pipeline again. It is the human-review step the GAP analysis keeps on purpose: the person stays in control.

The loop has three moving parts: an assistant that suggests what to fix, a router that decides which agents need to re-run, and a stateless round-trip that carries the whole campaign with each request.



See the picture: [diagrams/refinement-routing.mmd](#)

THE PARTS

1. Proactive suggestion. When the result page loads, it calls `/suggest`. The assistant reads the finished campaign — persona, both variants, CTA scores, flag count — and returns two or three plain-text improvements, each one sentence. It fires once, on load, so the user is told what to look at first instead of having to know what to ask. This mirrors an account manager reviewing creative before the client does.

2. The sliders and the chat. The panel offers three sliders — tone, audience age, formality — and a free-text box. Sliders are deterministic and instant to read; chat is for requests that need reasoning. The sliders sit above the chat on purpose: they are the cheap, reversible thing to try first.

3. The router. On `/refine`, `assistant.get_routing` looks at which sliders moved from their defaults (3 / 35 / 3) and at the user's message, then decides the smallest set of agents to re-run.

HOW THE ROUTING WORKS

The router follows fixed rules, not guesswork:

- A tone or formality change re-runs copywriter, guardrails, cta_optimizer.
- An age change also re-runs audience (the persona itself shifts).
- A request about wording re-runs copywriter, guardrails, cta_optimizer.
- A request only about the call to action re-runs cta_optimizer alone.
- Anything unclear falls back to copywriter, guardrails, cta_optimizer.

Two safety rails sit on top of the model's answer. Agent names are checked against a whitelist, and the list is re-sorted into pipeline order. If the copywriter is going to run, guardrails is forced to run straight after it — new copy is never shown without a fresh compliance check. The router never re-runs vision or color, because the product image did not change.

WHY THE ROUND-TRIP IS STATELESS

Every /refine request carries the full output_state of the campaign, and the server sends the merged state back. The server keeps nothing between requests. This was not the original plan — it came from wanting the tool to run without session storage or a login. The cost is a larger request body; the benefit is that any page can refine any campaign it is holding, and a restart loses nothing that the page already has.

HOW SLIDERS BECOME PROMPTS

map_sliders_to_params turns the raw slider integers into agent-ready text. Tone 1–5 maps to casual, playful, professional, emotional, energetic. The age value becomes a range of roughly plus-or-minus seven years, fed to the audience agent as a target. Formality 1–5 maps to an instruction string from "very casual, use slang" up to "highly formal, corporate register", appended to the copywriter's tone.

WHAT HAPPENS IF SOMETHING BREAKS

Both assistant calls fail soft. If /suggest cannot reach the model, it returns two generic but useful tips instead of an error. If the router's model call or JSON parse fails, it falls back to the safe default set (copywriter, guardrails, cta_optimizer) so a refine always does something sensible. Only the agents that ran are merged back; the rest of the report is left untouched.

WHY IT'S BUILT THIS WAY

The visible routing label is a direct answer to the brief's "make the collaboration visible" requirement — the user sees which agents ran on each refine, not just that the output changed. Re-running a subset instead of the whole pipeline keeps refinement fast enough to iterate, and the forced copywriter-then-guardrails order keeps compliance from ever being skipped. This loop is the subject of DL-05.

WHERE THIS LIVES IN THE CODE

- agents/assistant.py suggest, get_routing, map_sliders_to_params

- app.py /suggest and /refine endpoints, state merge
- templates/result.html the two-column page, sliders, chat, routing label

DIAGRAMS FOR THIS DOCUMENT

- diagrams/refinement-routing.mmd one refine, as a sequence